

Data-Driven Analysis and Prediction of Shipment Delays in E-commerce Supply Chains

Name: Smit Madat (snm178) | Git: <https://github.com/Smitmadat16/Shipment-Delay-Prediction> |

Video: <https://youtu.be/Bs184gh1lpc>

1. Project Definition

1.1 Problem Statement

Online shopping has skyrocketed in recent years and especially after the pandemic year, people started ordering everything online. Everyday a lot of people are placing amplex of orders from different states and regions and as a result it is always very difficult to hand over the orders to the customers on time because of which customers become irritated and get angry. However there are several reasons or factors that cause delay in delivering their orders such as how far it is coming from, number of orders, traffic, and so on. Hence because of all these, business as a whole suffers a lot as customers sometimes ask for refunds and even cancel their orders midway which cause a great loss in business. Hence it would be better if customers get to know the reason why order is being delayed and predict by how many days it's going to be delayed before they happen. So this project is about predicting shipment delays and why they happen. I will use a real dataset (supply chain) which includes 180,519 records to figure out the reason for delays and hence help us to make a model which can predict whether shipment will be late or not.

1.2 Connection to Course Material

This project is connected with the material that we covered in our class throughout the semester. It covers almost all the topics we did in our class such as Data cleaning, analysis and so on. First of all, I started by collecting data from google where I found a real data set of the supply chain. Later on using that data set, I started with cleaning data, fixing data formats, eliminating duplicates, and handling missing values using pandas. After that I used Feature engineering to turn old columns into new which can be useful for further analysis. Later on, to comprehend what was happening in data, I did data analysis using visualizations which gave me a rough idea of what's going on in this dataset. I used SQL queries to examine different kinds of trends based on region , month , year, and so on. Finally, I used Scikit-learn to train models using Machine learning where I used different methods such as Random forest, Logistic Regression, and Decision Tree to predict delays. Hence this project will cover all the topics starting from data loading , cleaning all the way to Machine Learning.

2. Novelty and Importance

2.1 Importance of the project

Shipment delays are the major concern for every online retailer nowadays as they are too expensive and even make customers dissatisfied. In today's contemporary world, Most of the

businesses are delivering their products online and almost all of them have to suffer from this problem and have to spend a lot of money attempting to solve logistical problems, However they get to know about the delays after they occur. So if they get to know about the delay before they occur then they might take some early action. In order to know this before, this initiative uses data to anticipate whether shipment would be delayed based on operational criteria such as order timing, product category, delivery mode, and geographical location. Hence this will be very helpful for businesses as they will be able to work on it before delay occurs.

2.2 Excitement and Relevance

Shipment delays are a real world problem and businesses like Amazon and FedEx have to deal with this problem everyday. I was really very interested in this project as it feels like solving real world problems and not just like working on any class project for getting good grades. It was an amazing experience to solve such a problem from scratch, from cleaning, finding missing values to a Web application that makes predictions. It was something like I am working on a real data science project.

2.3 Review of Related Work

The fast expansion of e-commerce and online delivery services has made supply chain analytics and shipping delay prediction significantly research areas. Prior research used machine learning techniques to solve logistics issues. They used commonly approached methods like Random Forest, decision trees and so on. Whereas few studies now in existence integrate machine learning and many studies focus on accuracy. For better understanding reasons behind shipment delays and also to predict them beforehand, this project is really helpful as it uses a combination of SQL analysis , data visualization and modeling. The DataCo Smart Supply Chain dataset was chosen for end-to-end supply chain research as it includes a variety of operational, customer, product, and shipping related factors.

3. Progress and Contribution

3.1 Data Utilization

I used the DataCo Smart Supply Chain dataset, which comprises 180,519 rows and around 53 columns covering everything from product classifications and customer locations to order dates and shipping methods. I imported it into a SQLite database to understand different trends and to keep it structured. I started by loading and inspecting data using pandas, cleaning data by finding and eliminating missing values, removing irrelevant data and changing date columns to the correct datetime format. Further moving to Feature Engineering, I formed a variable 'delay' by comparing actual shipping days to scheduled shipping data and if real days exceeded schedule days then it was considered as delayed.

Cleaning Data

```
df_cleaned = df.copy()
df_cleaned = df_cleaned.drop(columns=['Product Description', 'Order Zipcode'])

print("Original shape:", df.shape)
print("Cleaned shape:", df_cleaned.shape)
```

Original shape: (180519, 53)
Cleaned shape: (180519, 51)

```
df_cleaned = df_cleaned.dropna(subset=['Customer Lname', 'Customer Zipcode'])
print("Shape after handling small missing values:", df_cleaned.shape)
```

Shape after handling small missing values: (180508, 51)

Removing Irrelevant Data

```
df_cleaned = df_cleaned.drop(columns=[
    'Customer Email',
    'Customer Password',
    'Customer Fname',
    'Customer Lname',
    'Customer Street',
    'Product Image'
])
print("Shape after removing irrelevant columns:", df_cleaned.shape)
```

Shape after removing irrelevant columns: (180508, 45)

Missing Data

```
missing_values = df.isnull().sum()
total_rows = len(df)
missing_values_percent = (missing_values / total_rows) * 100
missing_info = pd.DataFrame({
    'Missing Value in each column': missing_values,
    'Percentage of missing values (%)': missing_values_percent
})
print(missing_info)
```

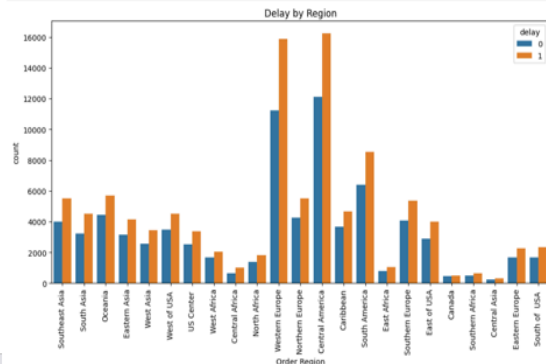
Then, I tried finding different trends by using week , month and year , to find out if shipment delay was consistent or same all time or not. I even tried visualizing the delay by month , region , year and shipping mode to get a clear idea of what the trend is following.

Target Variable Creation

```
df_cleaned['delay'] = (
    df_cleaned['Days for shipping (real)'] >
    df_cleaned['Days for shipment (scheduled)']
).astype(int)
print(df_cleaned['delay'].value_counts())
```

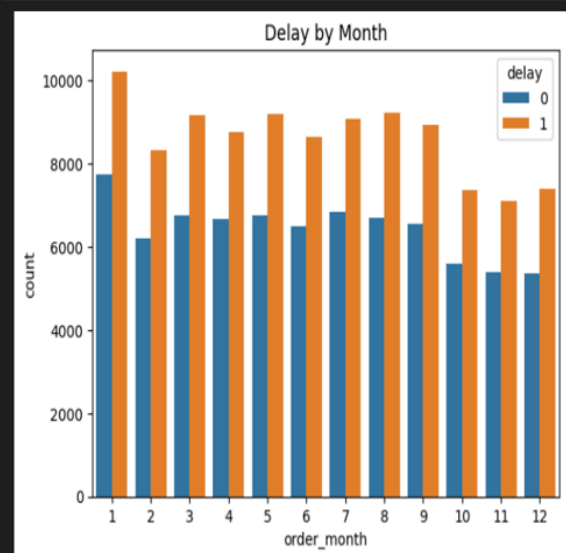
Delay By Region

```
plt.figure(figsize=(12,6))
sns.countplot(x='Order Region', hue='delay', data=df_cleaned)
plt.title("Delay by Region")
plt.xticks(rotation=90)
plt.show()
```



Delay By Month

```
sns.countplot(x='order_month', hue='delay', data=df_cleaned)
plt.title("Delay by Month")
plt.show()
```



3.2 Models, Techniques, and Algorithms

I made extensive use of SQL to get different trends and to get information about consistency of delaying shipment. I created queries to determine monthly trends, product categories with the greatest delay rates, top regions with most delays, and shipping mode specific delay rates. After that I even did visualization for all of them to know trends better. I used a horizontal bar chart for shipping mode and regions, a lollipop plot for product categories, and a shaded line chart for monthly trends.



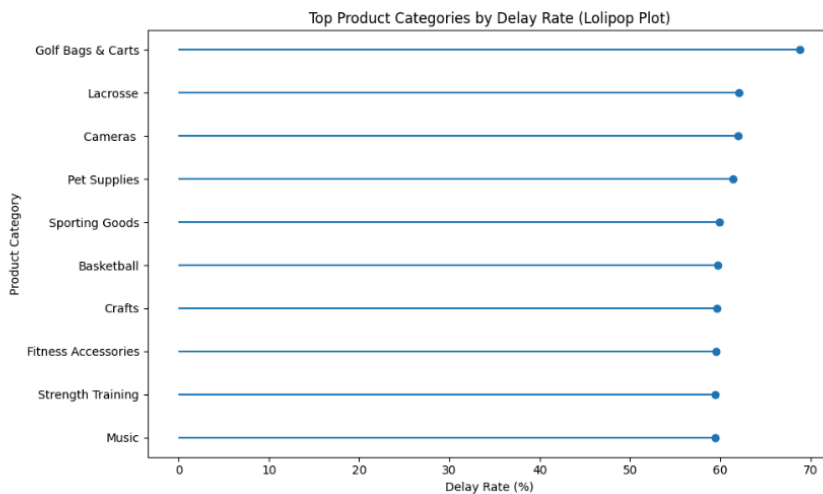
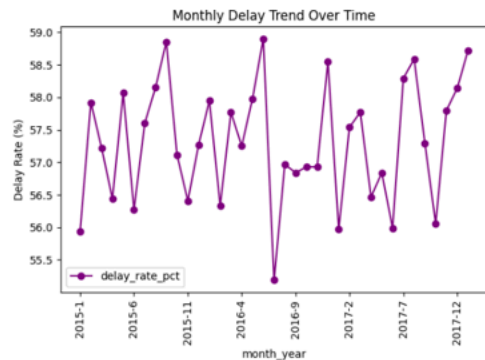
Delay Shipping Mode

```
query = """
SELECT [Shipping Mode],
COUNT(*) AS total_orders,
SUM(delay) AS delayed_orders,
AVG(delay) AS delay_rate
FROM orders
GROUP BY [Shipping Mode];
"""
result = pd.read_sql(query, conn)
print(result)
```

	Shipping Mode	total_orders	delayed_orders	delay_rate
0	First Class	27812	27812	1.000000
1	Same Day	9737	4657	0.478279
2	Second Class	35214	28077	0.797325
3	Standard Class	107745	42849	0.397689

Monthly Delay Trend Over Time

```
query = """
SELECT order_year,
order_month,
COUNT(*) AS total_orders,
ROUND(AVG(delay) * 100, 2) AS delay_rate_pct
FROM orders
GROUP BY order_year, order_month
ORDER BY order_year, order_month;
"""
result = pd.read_sql(query, conn)
print("Monthly Delay Trend:")
print(result)
```



Then I trained three models: Decision Tree to identify non-linear patterns, logistic regression for straightforward baseline and lastly Random Forest as the primary model. However in performing all these tasks, I forgot about data leakage in the middle of the project. The model appeared much better than it actually was because I had unintentionally included columns that were closely related to the target variable. All of sudden, I realized and I made it correct by eliminating those columns and retrained everything. Later on, in order to ensure that the model was stable over several data splits, I also used Grid SearchCV to adjust the Random Forest parameters for the Machine Learning portion and performed cross validation.

Model Comparison

```
print("Logistic Recursion Accuracy:", accuracy_score(y_test, y_pred_lr))
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
```

```
Logistic Recursion Accuracy: 0.6996565287241704
Decision Tree Accuracy: 0.7797905933189297
Random Forest Accuracy: 0.7237826159215556
```

3.3 Experimental Design

I anticipated that shipping mode would be the most important element in predicting delays as compared to other elements because of extremely varying delivery expectations. Apart from this, regions and product categories also play vital roles in determining shipment delay. The features were first scaled and data was divided in 80/20 for testing and training. Then later on Model training took place using three different models.

Train-Test Split

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

```
Train shape: (144406, 129)
Test shape: (36102, 129)
```

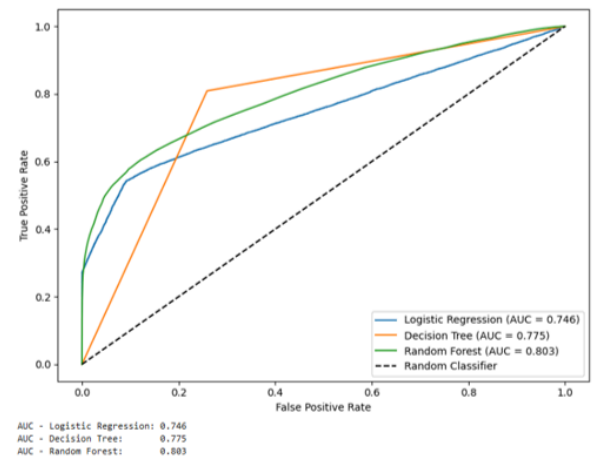
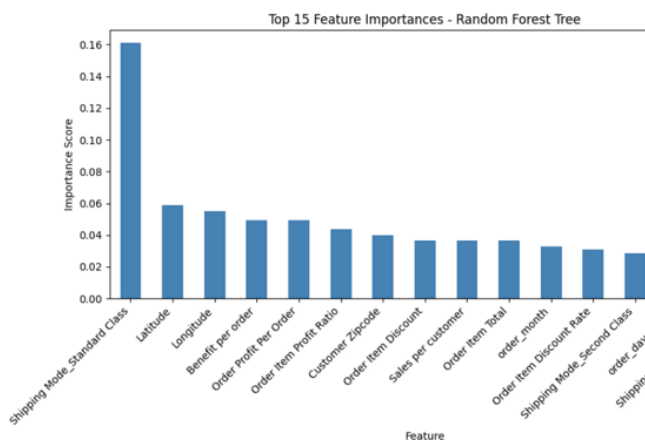
3.4 Key Findings and Results

The outcomes were quite intriguing. I found that all the first class got a delay rate of 1.000 which is 100%, whereas the same day got a delay rate of 47.82% and the second class and standard class got delay rate of 79.73% and 39.76% respectively getting to conclusion that Faster shipping choices generally have greater delay rates which can be because of tighter delivery

deadlines. I found out that delay rates were consistent over time and across different locations , so delays are more of an operational problem rather than a geographical or seasonal problem. Also Golf Bags and Carts had the greatest delay rate among product categories which is at 68.85%. Hence as a result, I would go for shipping mode as the most significant feature which supports my initial idea. The three models we used produced an Accuracy as follows :

1. Logistic Regression : 0.6996 almost 69.96%
2. Decision Tree : 0.7797 almost 77.98%
3. Random Forest: 0.7237 almost 72.37%

Based on this , I first thought Decision Tree is best for my project, However further using ROC curve which tells how well the model perform at various categorization criteria and AUC - Area under the Curve, I found that AUC for LR is 0.746 , for Decision Tree is 0.775, for Random Forest is 0.803. Hence Random Forest having highest AUC indicating superior overall classification ability, I chose Random Forest as my Ultimate model.



3.5 Evaluation

Later, I used AUC, Confusion Matrix, accuracy, and cross-validation to evaluate the models. Here is comparison:

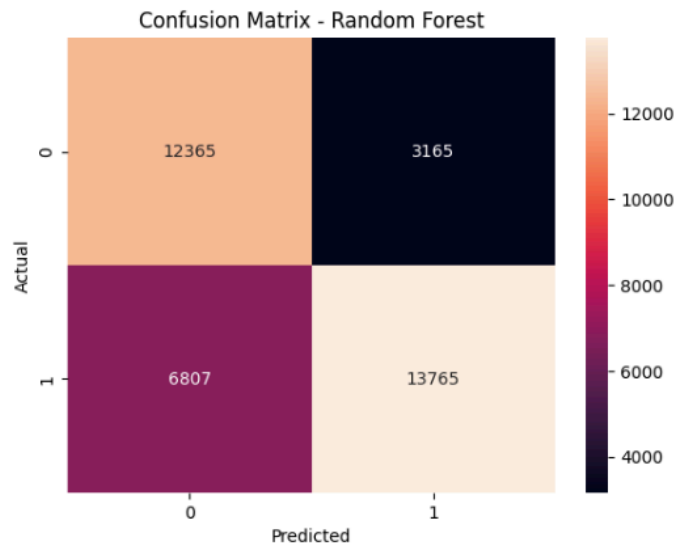
Model	Accuracy	AUC
Logistic Regression	0.700	0.746
Decision Tree	0.780	0.775
Random Forest	0.724	0.803

AUC is more reliable than accuracy because it measures how well the model separates delayed vs on time shipments.

Confusion Matrix

```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred_rf)
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix - Random Forest")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



3.6 Advantages and Limitations

In my opinion, the greatest advantage of this project is that it is covering an entire pipeline from start till end. However there are undoubtedly restrictions. External variables that could impact delays, like weather conditions are not included in the dataset. Apart from this 100% delay rate for first class doesn't seem true, there could be a problem with original datasets. Additionally, accuracy of random forest was low as compared to decision trees which indicate that the dataset may not be complex enough to fully benefit.

4. Changes after Proposal

4.1 Differences from Proposal

I mentioned in the proposal that I am going to use 50k records. At that time I thought 180,519 would be too large for performance but however when I actually tried using this large dataset, performance was not a concern. PostgreSQL was also included as a database option in

the proposal, but since SQLite is easier to set up and works directly with python, I chose it instead. Everything else remained as suggested, including models, analysis methodology, and the dataset.

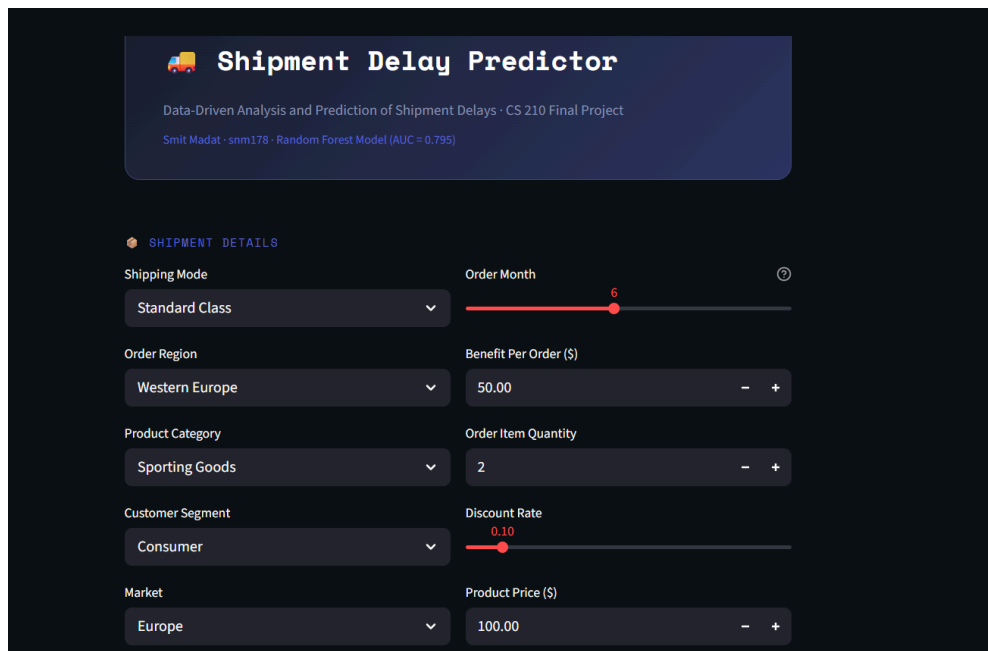
4.2 Bottlenecks and Challenges

The biggest challenge I faced was the data leak as I mentioned earlier, I wasn't immediately aware of it and spent a lot of time thinking about why my model is doing so well. It was so annoying for me as I could not figure it out quickly and it took me a long time to figure it out, but finally was able to do so and made corrections immediately. Another challenge was about the Random Forest performed with less accuracy as compared to Decision tree, but after spending some time, I realized that a single tree is sufficient to represent the patterns without the requirement for an ensemble because the data just has rather simple decision bounds.

5. Conclusion and Future Work

5.1 Contributions

My project developed a comprehensive data science pipeline starting from scratch by finding data sets , cleaning , visualizing and storing it in a SQLite database. Then I ran different queries, found different trends and compared three learning models, and implemented the best model in real time prediction Streamlit app. It was really interesting putting everything together from start to end.



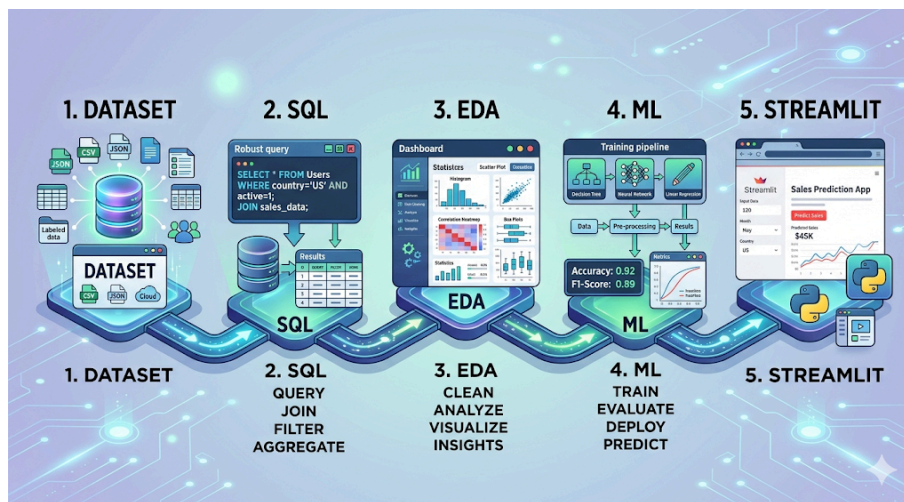
The screenshot displays the 'Shipment Delay Predictor' application interface. At the top, a header bar contains a truck icon, the title 'Shipment Delay Predictor', and a subtitle 'Data-Driven Analysis and Prediction of Shipment Delays - CS 210 Final Project'. Below the subtitle, it credits 'Smit Madat - snm178 - Random Forest Model (AUC = 0.795)'. The main interface is titled 'SHIPMENT DETAILS' and features a grid of input controls. On the left side, there are four dropdown menus: 'Shipping Mode' (set to 'Standard Class'), 'Order Region' (set to 'Western Europe'), 'Product Category' (set to 'Sporting Goods'), and 'Customer Segment' (set to 'Consumer'). On the right side, there are two sliders and two numeric input fields. The 'Order Month' slider is set to 6, and the 'Discount Rate' slider is set to 0.10. The 'Benefit Per Order (\$)' field is set to 50.00, and the 'Product Price (\$)' field is set to 100.00. Each numeric field has minus and plus buttons for adjustment.

5.2 Future Directions

Weather and Holidays undoubtedly have an impact on shipping, so adding such external data could improve the model's accuracy and therefore there are definitely requirements for improving this project. I was not able to fully solve it for sure because it can be made more accurate. Additionally, it would be great to make the Streamlit app available online so that anyone could use it without having to run it locally.

- Pipeline of entire project looks somewhat like this:

Dataset → SQL → EDA → ML → Streamlit



Citation:

Constante, F., Silva, F., & Pereira, A. (2019). DataCo Smart Supply Chain for Big Data Analysis [Dataset]. Mendeley Data.

<https://data.mendeley.com/datasets/8gx2fvg2k6/5>

